Edu SaaS Backend API Documentation

Generated from the repository code on 2026-06-22. This documentation covers endpoints mounted in `src/app.js`, `src/routes/authRoutes.js`, `src/routes/institutionRoutes.js`, and `src/routes/platformRoutes.js`. When exact behavior is broad or product-dependent, it is marked TODO/VERIFY instead of guessed.

Overview

This is a Node.js + Express + Mongoose multi-tenant Edu SaaS backend for school and university institutions.

Mounted API surface:

- `POST /stripe/webhook` is mounted before JSON parsing and uses `express.raw({ type: "application/json" })`.
- `/auth` is mounted from `src/routes/authRoutes.js`.
- `/institutions` is mounted from `src/routes/institutionRoutes.js`.
- `/platform` is mounted from `src/routes/platformRoutes.js` as the separate provider namespace.
- `/uploads` is static public storage for institution logos and member profile pictures only.

Route files inspected:

- `src/app.js`
- `src/server.js`
- `src/routes/authRoutes.js`
- `src/routes/institutionRoutes.js`
- `src/routes/platformRoutes.js`

Supporting code inspected:

- Controllers, services, middleware, permission catalogs, scoped responsibility helpers, models, upload middleware, tests, Postman files, package/env references, and error handlers.

Authentication Flow

JWT authentication uses `Authorization: Bearer <JWT_TOKEN>`.

`authJwt`:

- Reads the bearer token.

- Verifies it with `JWT_SECRET`.
- Requires `payload.sub` to be a valid user id.
- Loads the user and rejects missing or inactive accounts.
- Sets `req.user = { id, email, status, isEmailVerified }`.

`requireEmailVerified`:

- Requires `req.user`.
- Rejects unverified accounts with `403 { "message": "Email is not verified" }`.

Auth flow:

1. `POST /auth/signup` creates a user, institution, admin membership, sends signup OTP, and returns a JWT.
2. Signup-created users are not fully usable for tenant routes until OTP verification.
3. `POST /auth/otp/verify` verifies email for the authenticated user.
4. `POST /auth/login` requires an active verified email and returns a JWT.
5. Forgot/reset password uses OTP and generic forgot-password messaging.

Tenant / Institution Scoping Rules

Tenant-owned business routes live under `/institutions/:institutionId/...`.

`institutionAccess()`:

- Reads `req.params.institutionId`, falling back to `x-institution-id` only when no path param exists.
- Requires the authenticated user to have an active membership in that institution.
- Sets `req.tenantId` and `req.membership`.

Scoping rules:

- Institution-owned data must be queried with `institutionId` plus resource ids.
- Do not fetch/update/delete tenant-owned records by `_id` alone.
- `GET /institutions/me` and `POST /institutions` are authenticated account-level routes and do not use a tenant path param.
- Public/global endpoints are auth signup/login/password flows and the Stripe webhook.

Role And Permission Model

`ROLE_PERMISSIONS` from `src/auth/permissions.js`:

Role	Permissions
admin	institution.manage, department.manage, class.manage, membership.manage, semester.manage, subject.manage, section.read, section.manage, student.kick, attendance.manage, billing.manage, academicDecision.review, academicDecision.apply
instructor	section.read, attendance.take, attendance.excuse
student	section.read, enrollment.self_manage, attendance.read_self, billing.read_self

Notes:

- `Membership.role` values are `admin`, `instructor`, and `student`.
- Institution ownership is checked in selected services in addition to membership role.
- Some section-linked routes rely on service-level authorization instead of route-level `requirePermission(...)`.
- Students have `section.read` for student-facing section-linked flows, but the generic staff section list/detail service blocks student memberships.

Responsibility / Scoped Permission Model

Responsibilities are scoped assignments, not `Membership.role` values. The backend catalog is the source of truth for effective responsibility permissions; raw stored `ResponsibilityAssignment.permissions` are not trusted for authorization.

`RESPONSIBILITY_PERMISSIONS`:

Responsibility	Effective permissions
head	academicRisk.read_scope, academicProfile.read_scope, academicStanding.read_scope, academicDecision.read_scope, academicDecision.recommend_scope
coordinator	section.read_scope, sectionSession.read_scope, sectionSession.manage_scope, attendance.read_scope, attendance.manage_scope
mentor	none currently active

Allowed responsibility scopes:

Responsibility	Allowed `scopeType` values
head	institution, department, classLevel
coordinator	department, classLevel, class, section
mentor	class, section

Assignments must be active, within optional `startAt` / `endAt`, same tenant, and valid for their responsibility/scope combination.

Head assignments provide read-only academic oversight plus scoped AcademicDecision read/recommend behavior only. Head does not receive section/session/attendance/enrollment/assessment/result/billing/membership write or operational permissions, and cannot approve/reject/apply decisions.

Coordinator assignments are currently used by selected section reads, section-session reads/management, and attendance reads/management.

Mentor assignments have no effective operational or academic permissions yet.

Do not activate future scoped permissions such as `studentProfile.read_scope`, `attendance.excuse_manage_scope`, `academicDecision.review_scope`, `enrollment.review_scope`, or `studentDropRisk.review_scope` until service-level enforcement exists.

Platform Provider Control Plane APIs

The `/platform` namespace is a provider-level, read-only control plane. Platform authority is separate from tenant membership:

```
User -> PlatformAdmin -> platform role -> platform permissions
```

An active, email-verified User with an active `PlatformAdmin` record can use these APIs without a tenant Membership. Conversely, an institution owner or tenant admin receives no platform access merely from their Membership.

Platform role and permissions

Phase 1 supports one platform role:

Role	Permissions
`platform_owner`	`platform.institutions.read`, `platform.billing.read`, `platform.webhooks.read`, `platform.audit.read`

Every platform request passes through this sequence:

1. `authJwt` verifies the bearer JWT and loads an active User.
2. `requireEmailVerified` rejects unverified Users.
3. `resolvePlatformAdmin` loads an active `PlatformAdmin` for the same User and derives permissions from the backend role catalog.
4. `requirePlatformPermission(...)` checks the route-specific permission.
5. The service re-reads the active PlatformAdmin and permission catalog before accessing provider data.
6. The operation writes a sanitized `PlatformAuditLog` success or failure record. Permission denials are audited on a best-effort basis.

Platform routes never use `institutionAccess()` and never trust a tenant Membership as provider authority.

Platform owner setup and frontend flow

There is intentionally no HTTP endpoint for granting platform authority. Create the first owner through the trusted CLI after the User exists and is active and email-verified:

```
npm run platform:create-owner -- --email owner@example.test
```

Recommended client/operations flow:

1. Seed or create the active verified User.
2. Run `platform:create-owner` once. It refuses missing/ineligible Users and duplicate PlatformAdmin records.
3. Log in through `POST /auth/login` with the platform owner's normal User credentials.
4. Store the returned `accessToken` and send `Authorization: Bearer <accessToken>` to `/platform/...`.
5. Load `GET /platform/institutions` for the control-plane landing page.
6. Open `GET /platform/institutions/:institutionId` for institution operations detail.
7. Open `/billing`, `/webhook-events`, or `/audit` only for the corresponding operational screen.
8. Treat `401` as an invalid/expired login and `403` as missing platform authority or permission. Do not fall back to tenant-admin authorization.

Local Postman flow is documented in `postman/PLATFORM_CONTROL_PLANE_POSTMAN_GUIDE.md` and uses:

- `postman/platform-control-plane.postman_collection.json`
- `postman/platform-control-plane.local.postman_environment.json`

Platform endpoint summary

All endpoints require `Authorization: Bearer <accessToken>`. Pagination defaults to `page=1&limit=25`; `limit` must be between 1 and 100. Unexpected query fields return `400`.

Method	Path	Permission	Query fields	Response
GET	`/platform/institutions`	`platform.institutions.read`	`page`, `limit`, `status`, `type`, `search`, `plan`, `subscriptionStatus`	`{ items, pagination }`
GET	`/platform/institutions/:institutionId`	`platform.institutions.read`	None	`{ institution, counts, billing?, seatUsage?, billingSync? }`
GET	`/platform/institutions/:institutionId/billing`	`platform.billing.read`	`page`, `limit`, `eventType`, `status`, `stripeEventId`, `dateFrom`, `dateTo`	`{ institutionId, billing, seatUsage, billingSync?, failedWebhooks? }`
GET	`/platform/webhook-events`	`platform.webhooks.read`	`page`, `limit`, `eventType`, `status`, `institutionId`, `stripeEventId`, `dateFrom`, `dateTo`	`{ items, pagination }`
GET	`/platform/audit`	`platform.audit.read`	`page`, `limit`, `actorPlatformAdminId`, `action`, `targetType`, `targetId`, `institutionId`, `result`, `dateFrom`, `dateTo`	`{ items, pagination }`

Allowed filter enums:

- Institution `status`: `active`, `archived`.

- Institution `type`: `school`, `university`.
- Billing `plan`: `plus`, `pro`.
- Subscription status: `incomplete`, `trialing`, `active`, `past_due`, `canceled`, `unpaid`, `incomplete_expired`, `paused`.
- Audit `result`: `success`, `denied`, `failed`.
- Webhook `status`, when supplied, must be `failed`; this API never exposes processed webhook payloads.

`dateFrom` and `dateTo` accept ISO dates/timestamps and are inclusive when a date-only `dateTo` is supplied. `dateFrom` must not be later than `dateTo`.

`GET /platform/institutions`

Returns safe institution identity and aggregate active-membership counts. Billing filters require `platform.billing.read`. The returned fields are permission-shaped: billing permission adds `billing` and `seatUsage`, while webhook permission adds `billingSync`.


```
{
  "items": [
    {
      "institutionId": "507f1f77bcf86cd799439011",
      "name": "Demo School",
      "type": "school",
      "status": "active",
      "counts": {
        "totalMemberships": 9,
        "activeMemberships": 9,
        "activeStudents": 5,
        "activeStaff": 4
      },
      "billing": {
        "plan": "pro",
        "billingInterval": "six_month",
        "subscriptionStatus": "trialing",
        "hasPaidAccess": true,
        "currentPeriodStart": "2026-04-01T00:00:00.000Z",
        "currentPeriodEnd": "2026-10-01T00:00:00.000Z",
        "cancelAtPeriodEnd": false,
        "trialEnd": "2026-09-15T00:00:00.000Z"
      },
      "seatUsage": {
        "activeStudents": 5,
        "maxActiveStudents": 1000,
        "activeStaff": 4,
        "maxActiveStaff": 100
      },
      "billingSync": {
        "state": "stale",
        "lastWebhookAt": null,
        "failedWebhookCount": 0
```

```
      }
    }
  ],
  "pagination": { "page": 1, "limit": 25, "total": 1, "totalPages": 1 }
}
```

`GET /platform/institutions/:institutionId`

Returns institution timestamps and canonical membership counts: total Users/Memberships, active/inactive/suspended Memberships, active Students/Admins/Instructors, and combined active Staff. With billing permission it also returns billing, full seat availability, and Stripe customer/subscription/price IDs. With webhook permission it returns full billing synchronization state.

A missing institution returns `404 { "message": "Institution not found" }`; an invalid ID returns `400`.

`GET /platform/institutions/:institutionId/billing`

Returns the local webhook-synchronized billing projection and canonical seat usage. The route parameter is the only institution source; an `institutionId` query field is rejected.

If the actor also has `platform.webhooks.read`, the response includes `billingSync` and a paginated `failedWebhooks` result filtered to this institution. Without that permission, both fields are omitted. Stripe secret keys, client secrets, raw webhook payloads, and arbitrary Stripe objects are never returned.

Billing synchronization `state` values:

State	Meaning
`synced`	Billing mappings are recognized and a successful correlated webhook occurred within seven days.
`missing_profile`	No local institution billing profile exists.
`missing_customer_mapping`	The profile has no Stripe customer ID.
`missing_subscription_mapping`	The profile has no Stripe subscription ID.
`unknown_plan`	The local plan is missing or unsupported.
`unknown_price`	The Stripe price is missing or does not resolve to the local plan.
`webhook_failed`	At least one correlated failed webhook exists.
`stale`	No successful correlated webhook exists within seven days.
`unknown`	No more specific state could be derived.

Stripe remains authoritative. These APIs report local synchronized state and do not perform live Stripe reconciliation.

`GET /platform/webhook-events`

Returns only failed Stripe webhook summaries. Each item contains `webhookEventId`, `stripeEventId`, `eventType`, `status`, nullable `institutionId`, a sanitized/truncated `errorMessage`, `attemptCount`, timestamps, and nullable `processedAt`.

Historical failures without institution correlation remain global with `institutionId: null`; the backend does not guess a tenant. Raw webhook payloads and secrets are never exposed. A clean environment may legitimately return an empty list.

`GET /platform/audit`

Returns immutable, allowlisted provider audit records. Each item contains actor IDs, action, target fields, nullable institution/reason, safe metadata, IP address, user agent, result, and creation time.

Current audit actions are:

- `platform.institutions.list`
- `platform.institutions.view`
- `platform.institutions.billing.view`
- `platform.webhooks.failures.list`
- `platform.audit.list`
- `platform.permission.denied`

Audit metadata is restricted to `page`, `limit`, `filterCount`, `returnedCount`, and `errorCode`. Raw errors, stacks, bodies, tokens, passwords, and Stripe secrets are never written. An authorized operation failure attempts a sanitized `result=failed` audit with `errorCode=PLATFORM_OPERATION_FAILED` while preserving the original API error.

Phase 1 boundaries

The platform namespace is GET-only. It does not provide tenant or billing mutation, webhook replay, live reconciliation, institution suspension, impersonation/support access, or platform-role management. MFA is recommended production hardening but is not implemented by this phase.

Common Headers

Header	Required for	Notes
`Authorization: Bearer <JWT_TOKEN>`	Protected routes	Required by `authJwt`.
`Content-Type: application/json`	JSON request bodies	Default for create/update APIs.
`Content-Type: multipart/form-data`	Upload routes	See file notes below.
`Stripe-Signature`	`POST /stripe/webhook`	Required for Stripe signature verification.

Header	Required for	Notes
`x-institution-id`	Rare fallback	Supported by `institutionAccess()`, but current tenant routes use `:institutionId`.

Common Response Envelope Shape

Controllers return service envelopes directly. Common shapes are:

```
{ "resourceName": { "id": "507f1f77bcf86cd799439011" } }
```

```
{ "resources": [], "pagination": { "page": 1, "limit": 20, "total": 0, "totalPages": 0 } }
```

```
{ "message": "Operation completed successfully" }
```

Common envelope keys include `user`, `institution`, `membership`, `department`, `classLevel`, `schoolClass`, `assignment`, `subject`, `semester`, `academicYear`, `member`, `section`, `sectionSession`, `attendanceRecord`, `attendanceRecords`, `attendanceRoster`, `attendanceAnalytics`, `assessment`, `assessmentResults`, `homework`, `submission`, `material`, `academicPolicy`, `schoolAcademicPolicy`, `academicProfile`, `academicRiskSummary`, `reportCard`, `schoolFinalOutcome`, `promotionSummary`, `sectionOutcomeSummary`, `standingSummary`, `responsibilityAssignment`, `responsibilityAssignments`, `students`, `billingProfile`, and `checkoutSession`.

Common Error Response Shape

All route/controller error responses use:

```
{ "message": "..." }
```

Common statuses:

Status	Meaning
400	Invalid ObjectId/body/query/upload, missing required field, invalid lifecycle field.
401	Missing/invalid/expired JWT, invalid credentials, unauthorized context.
403	Inactive account, unverified email, no tenant access, permission denied.
404	Tenant-scoped resource or active context not found.
409	Duplicate/conflict, invalid state transition, full capacity, already open/closed, wrong institution type.
429	OTP max attempts exceeded.
500	Internal errors; message is masked as `Internal server error`.
502	Stripe checkout/session failure.

File Upload / Download Notes

Use case	Field name	Storage	Accepted types	Size limit	Public
Institution logo	`logo`	`uploads/institution-logos`	image/jpeg, image/png, image/webp	Default 2MB, `MAX_LOGO_SIZE_MB` override	Yes
Member profile picture	`profilePicture`	`uploads/member-profiles`	image/jpeg, image/png, image/webp	Default 2MB, `MAX_PROFILE_PICTURE_SIZE_MB` override	Yes
Homework submission	`submissionFile`	`storage/homework-submissions`	application/pdf plus `.pdf` extension	Default 10MB, `MAX_HOMEWORK_SUBMISSION_SIZE_MB` override	No
Section material	`materialFile`	`storage/section-materials`	application/pdf plus `.pdf` extension	Default 10MB, `MAX_SECTION_MATERIAL_SIZE_MB` override	No

Private files are served only through protected download routes after service-level authorization. `storage/` is not exposed by `express.static`.

TODO/VERIFY: Current PDF validation is MIME/extension and service-level metadata/path safety; PDF magic-byte validation is deferred.

Environment / Base URL Notes

Default local base URL is `http://localhost:5000` unless `PORT` is set.

Environment variable names found in code/docs:

- `MONGO_URI`, `PORT`, `NODE_ENV`
- `JWT_SECRET`, `JWT_EXPIRES_IN`, `BCRYPT_SALT_ROUNDS`
- `OTP_EXPIRES_MINUTES`, `OTP_MAX_ATTEMPTS`
- `GMAIL_USER`, `GMAIL_APP_PASSWORD`, `EMAIL_FROM`
- `STRIPE_SECRET_KEY`, `STRIPE_PRICE_PLUS_6_MONTH`, `STRIPE_PRICE_PRO_6_MONTH`, `STRIPE_WEBHOOK_SECRET`, `BILLING_SUCCESS_URL`, `BILLING_CANCEL_URL`
- `MAX_LOGO_SIZE_MB`, `MAX_PROFILE_PICTURE_SIZE_MB`, `MAX_HOMEWORK_SUBMISSION_SIZE_MB`, `MAX_SECTION_MATERIAL_SIZE_MB`
- `ATTENDANCE_QR_DATE_OVERRIDE` for non-production local attendance QR testing only

Do not document or expose real secret values.

Full Endpoint Reference

See API_ENDPOINT_INDEX.md for the compact all-endpoint table. The grouped reference below adds request, validation, response, security, and TODO/VERIFY notes by module. All paths are full API paths.

Auth

Method	Path	Purpose	Auth	Request / validation	Success	Errors / notes
POST	`/auth/signup`	Create user, institution, admin membership, send signup OTP.	Public	JSON `email`, `password`, `confirmPassword`, `institutionName`, `institutionType` (`school` or `university`). Password >= 8, email unique.	`201 { accessToken, user, institution, membership }`	`409` duplicate email; sends OTP; email must be verified before tenant routes.
POST	`/auth/login`	Login active verified user.	Public	JSON `email`, `password`.	`200 { accessToken, user }`	`401` invalid credentials; `403` inactive/unverified.
POST	`/auth/password/forgot`	Send password reset OTP.	Public	JSON `email`; email format validated.	`200 { message }`	Generic response for unknown/ineligible email.
POST	`/auth/password/reset`	Reset password using OTP.	Public	JSON `email`, `code`, `newPassword`.	`200 { message }`	OTP expiry/max attempts enforced; new password must differ from current.
POST	`/auth/otp/verify`	Verify signup OTP.	`authJwt`	JSON `code`.	`200 { message, user }`	Does not use `requireEmailVerified`; returns already-verified message if applicable.
POST	`/auth/otp/resend`	Resend signup OTP.	`authJwt`	No body.	`200 { message }`	`409` if already verified.

Institutions

Method	Path	Purpose	Auth / middleware	Request / validation	Success	Errors / notes
GET	`/institutions/me`	Current user institution context.	`authJwt`, `requireEmailVerified`	No params/body.	`200 { user, institutions, activeInstitutionId, onboarding }`	Global account context; not tenant-scoped by path. Institution rows are enriched for frontend bootstrap.
POST	`/institutions`	Create additional institution.	`authJwt`, `requireEmailVerified`	JSON `name`, `type` (`school` or `university`).	`201 { institution, membership }`	Creates active admin membership for same user.
PATCH	`/institutions/:institutionId`	Update institution.	`authJwt`, `requireEmailVerified`, `institutionAccess()`, `requirePermission("institution.manage")`	Path `institutionId`; JSON patch accepted by `institutionService`.	`200 { institution }`	Tenant-scoped. TODO/VERIFY exact patchable policy fields before generated clients.
POST	`/institutions/:institutionId/logo`	Upload institution logo.	Same plus `uploadInstitutionLogo`	Path `institutionId`; multipart field `logo`.	`200 { institution }`	Public file URL under `/uploads/institution-logos`; JPG/PNG/WEBP only.

`GET /institutions/me` frontend contract

The existing `/institutions/me` endpoint is enriched in place; there is no `/v1` route and no second `/me` endpoint.

Top-level response:

```
{
  user,
  institutions,
  activeInstitutionId,
  onboarding
}
```

Each `institutions[]` item includes:

```
{
  institutionId,
  membershipId,
  name,
  type,
  institutionStatus,
  role,              // "admin" | "instructor" | "student"
  membershipStatus,
  joinedAt,
  isOwner,
  memberProfile,     // null or limited profile summary
  permissions,       // base role permissions only
  scopedCapabilities,
  activeSemester,
  activeAcademicYear
}
```

Frontend rules:

- Keep `role` as one of `admin`, `instructor`, or `student`; ownership is shown with `isOwner`, not an `"owner"` role.
- Use `permissions` only for base institution-wide role permissions from `ROLE_PERMISSIONS`.
- Use `scopedCapabilities` for active Head, Coordinator, and Mentor powers. Do not flatten scoped permissions into global permissions.
- Show Head/Coordinator/Mentor labels from `scopedCapabilities[].responsibilityType`, not from `role`.
- `memberProfile` is `null` when no profile exists; when present it is limited to `fullName`, `firstName`, `lastName`, `mobile`, and `profilePictureUrl`.
- `activeSemester` and `activeAcademicYear` are read-only summaries and may be `null`.
- Use `activeAcademicYear` from `/institutions/me` for initial multi-institution bootstrap; use `GET /institutions/:institutionId/academic-years/active` to refresh the selected institution's current year.
- `onboarding.nextStep` is navigation guidance only, not authorization.
- Active non-admin memberships are usable access. Instructors, students, Heads, Coordinators, and Mentors complete onboarding with `nextStep: null`.
- `wait_for_membership_activation` means the user has memberships but none are active.
- `hasAdminMembership` means at least one active admin membership exists; it is informational and not required for access.

Billing / Stripe

Method	Path	Purpose	Auth / permission	Request	Success	Security notes
POST	`/institutions/:institutionId/billing/checkout-session`	Create Stripe-hosted subscription Checkout Session.	`institutionAccess()`; service requires owner or active admin.	`{ plan: "plus" }` or `{ plan: "pro" }` only.	`201 { checkoutSession: { id, url }, billingProfile, entitlements }`	Resolves Stripe Price IDs from server-side Plus/Pro env vars; rejects duplicate active/trialing/past_due subscriptions with subscription id.
POST	`/institutions/:institutionId/billing/upgrade-to-pro`	Immediately upgrade an existing Plus subscription to Pro.	`institutionAccess()`; service requires active owner or active admin.	No trusted body fields.	`200 { upgrade: { fromPlan, toPlan, status, clientSecret? } }`	Only Plus -> Pro is supported. Uses the existing Stripe subscription item, immediate proration, `payment_behavior=pending_if_incomplete`, and waits for verified webhook sync before granting Pro limits.
GET	`/institutions/:institutionId/billing/status`	Read local billing profile.	`institutionAccess()`; service requires owner or active admin.	No body.	`200 { billingProfile, entitlements }`	Returns local Stripe ids/status projection plus safe normalized entitlements; no card/payment method data.
POST	`/stripe/webhook`	Process Stripe webhook.	Public route, Stripe signature.	Raw `application/json` body with `Stripe-Signature`.	`200 { received, duplicate }`	Must remain before `express.json()`; verifies with `STRIPE_WEBHOOK_SECRET`; idempotent via `StripeWebhookEvent`.

Billing plan rules:

- `plus` and `pro` are six-month plans with the same application features.
- Plus uses `STRIPE_PRICE_PLUS_6_MONTH`, allows 300 active student memberships, and allows 30 active staff memberships.
- Pro uses `STRIPE_PRICE_PRO_6_MONTH`, allows 1000 active student memberships, and allows 100 active staff memberships.
- The backend plan catalog is authoritative. Clients never send Stripe Price IDs, Stripe Customer IDs, Stripe Subscription IDs, billing limits, paid status, or proration settings.
- Active student seats are active `Membership` records with `role: "student"`.
- Active staff seats are active `Membership` records with `role` in `["admin", "instructor"]`; owner admin memberships count as staff.
- `MemberProfile`, `Enrollment`, and `ResponsibilityAssignment` records do not consume seats.
- Paid access is granted only for known Plus/Pro plans with subscription status `active`, `trialing`, or `past_due`. `past_due` remains a temporary grace-period decision.
- Unknown or old Stripe prices keep `plan: null` / `billingInterval: null` and grant no paid access.

Plus -> Pro upgrade rules:

- The upgrade endpoint does not create a Checkout Session, Customer, or second subscription.
- The service validates local tenant billing state first: local plan must be `plus`, local subscription status must be `active`, local stored Stripe Price must match the configured Plus Price, and tenant Stripe Customer/Subscription IDs must exist.
- The service retrieves the live Stripe subscription and requires live status `active`, `collection_method: "charge_automatically"`, no subscription schedule, one subscription item, quantity `1`, and the configured Plus Price.
- The Stripe client is pinned to API version `2026-05-27.dahlia`. The update expands `latest_invoice.confirmation_secret`.

- Stripe update parameters are `items: [{ id: existingSubscriptionItemId, price: trustedProPriceId }]`, `proration_behavior: "always_invoice"`, and `payment_behavior: "pending_if_incomplete"`.
- A safe server-generated idempotency key includes institution id, subscription id, current latest-invoice marker, current Plus Price, and target Pro Price.
- The upgrade response is `processing` when Stripe has no pending update, `payment_action_required` only when trusted payment state requires action and a safe client secret exists, `payment_failed` for declined/missing-payment-method style pending updates, and `payment_incomplete` for other pending updates.
- Calling the upgrade endpoint never directly changes local `plan`, `stripePriceId`, subscription status, period dates, or seat limits.
- Pro limits become available only after a verified Stripe webhook synchronizes a trusted Pro Price from Stripe.
- `customer.subscription.created` and `customer.subscription.updated` webhooks retrieve the current Stripe subscription before updating the local billing profile, reducing stale-event regressions. `customer.subscription.deleted` applies the deletion snapshot.

Responsibility Assignments

Owner/admin management for scoped Head, Coordinator, and Mentor assignments. These routes use `authJwt`, `requireEmailVerified`, and `institutionAccess()`; the service performs the owner/admin check. A scoped Head/Coordinator/Mentor assignment alone cannot manage responsibility assignments.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/responsibility-assignments`	JSON `userId`, `responsibilityType`, `scopeType`, `scopeId`, optional `startAt`, `endAt`, `isActive`.	`201 { responsibilityAssignment }`	Owner/admin only. Target user must have active same-institution instructor membership.
GET	`/institutions/:institutionId/responsibility-assignments`	Query `userId`, `responsibilityType`, `scopeType`, `scopeId`, `isActive`, `activeNow`, `page`, `limit`.	`200 { responsibilityAssignments, pagination }`	Always scoped by route `institutionId`; `activeNow=true` or `activeNow=false` is evaluated by assignment status and date window.
GET	`/institutions/:institutionId/responsibility-assignments/:assignmentId`	Path `assignmentId`.	`200 { responsibilityAssignment }`	Tenant-scoped by `institutionId + assignmentId`.
PATCH	`/institutions/:institutionId/responsibility-assignments/:assignmentId`	JSON patch `scopeType`, `scopeId`, `startAt`, `endAt`, `isActive`.	`200 { responsibilityAssignment }`	`userId` and `responsibilityType` are not changed in V1; create a new assignment instead.
PATCH	`/institutions/:institutionId/responsibility-assignments/:assignmentId/deactivate`	No body.	`200 { responsibilityAssignment }`	Explicit soft-deactivation route for Head, Coordinator, and Mentor assignments.
DELETE	`/institutions/:institutionId/responsibility-assignments/:assignmentId`	No body.	`200 { responsibilityAssignment }`	Backward-compatible soft-deactivation by setting `isActive=false` and closing `endAt` when needed.

Create/update payload safety:

- Clients cannot set `permissions`, `institutionId`, `membershipId`, `createdByUserId`, `updatedByUserId`, `role`, `userId` on update, `responsibilityType` on update, or unknown authorization fields.

- `Membership.role` is never changed by these APIs. Head, Coordinator, and Mentor are assignment types only.
- Stored `ResponsibilityAssignment.permissions` is omitted from responses and ignored by authorization helpers.
- Duplicate active assignments are rejected for the same `institutionId + userId + responsibilityType + scopeType + scopeId`.

Frontend flow:

1. Load active institution context from `GET /institutions/me`.
2. For an admin/owner screen, list eligible instructor members with `GET /institutions/:institutionId/members?role=instructor&status=active`.
3. Let the admin pick responsibility type first, then only show allowed scopes for that type.
4. Load scope options from existing tenant routes: departments for `department`, class levels for `classLevel`, classes for `class`, sections for `section`, or use the current institution id for `institution`.
5. Submit only the allowed create fields. Do not send `permissions` or try to convert the member role.
6. After create/update/delete, refresh `GET /institutions/:institutionId/responsibility-assignments` with the same filters.

Backend flow:

1. `authJwt` validates the user token, then `requireEmailVerified` and `institutionAccess()` validate active tenant access.
2. Service verifies the actor membership is active and the actor is institution owner or base `Membership.role=admin`.
3. Service validates route `institutionId`, target active instructor membership, responsibility type, allowed scope combination, same-tenant scope record, date window, and duplicate-active conflicts.
4. Service writes only safe fields and stores an empty/non-authoritative `permissions` value.
5. Response returns safe assignment fields only: `id`, `institutionId`, `userId`, `responsibilityType`, `scopeType`, `scopeId`, `isActive`, `startAt`, `endAt`, `createdAt`, `updatedAt`.

Academic Oversight (Head Read-Only)

Read-only academic visibility for admin/owner and scoped Head assignments. These routes use `authJwt`, `requireEmailVerified`, and `institutionAccess()`; the service performs owner/admin or scoped Head authorization. Coordinator and Mentor assignments do not grant these academic oversight APIs.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/academic-oversight/risk-summary`	Optional narrowing filters `departmentId`, `classLevelId`, `sectionId`, `studentMemberId`, `search`, `riskLevel`, `page`, `limit`.	`200 { academicRiskSummary }`	Risk-only summary derived from attendance and published assessment results.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/academic-oversight/students`	Same safe narrowing filters.	`200 { academicRiskSummary, students, pagination }`	Returns risk-only compact student rows inside the actor's derived scope; no profile/standing fields.
GET	`/institutions/:institutionId/academic-oversight/students/:studentMemberId`	Path `studentMemberId`.	`200 { student }`	Profile detail route. Requires `academicProfile.read_scope`; returns `academicProfile` but not `academicStanding`.
GET	`/institutions/:institutionId/academic-oversight/students/:studentMemberId/standing`	Path `studentMemberId`.	`200 { student }`	Standing detail route. Requires `academicStanding.read_scope`; returns `academicStanding` but not `academicProfile`.

Head scope derivation:

- `institution` scope can see students/enrollments in the same institution.
- `department` scope is derived through `Subject.departmentId -> Section.subjectId -> Enrollment.studentMemberId`.
- `classLevel` scope is derived through `Subject.classLevelId` and `SchoolClass.classLevelId -> Section -> Enrollment`.
- Query filters only narrow the derived DB scope; they cannot expand it.

Academic risk derivation:

- Attendance risk uses only closed section sessions and attendance records from sections where the student has an active enrollment. Missing qualifying attendance records produce `level: "unknown"` rather than a false good state.
- Grade risk uses only published assessments from sections where the student has an active enrollment, plus matching assessment results. Missing published results and invalid assessment configuration are surfaced as risk reasons.
- Academic profile and standing are reused from existing academic services only on their dedicated detail routes. Missing optional data returns safe empty/null fields with `dataUnavailable` where appropriate.
- The `standing` query filter is not accepted on risk-only list/summary routes unless a route explicitly loads standing data; current risk endpoints return `403` for `standing=...`.

Frontend flow:

1. Use these routes only for admin/owner users or users with an active Head responsibility assignment.
2. Do not send a scope claim such as "I am department head of X" as authority. The backend reads the active `ResponsibilityAssignment` records and derives the allowed students.
3. Use filters for UI narrowing only, such as department, class level, section, risk level, or search.
4. Treat the response as read-only oversight. There are no actions here to drop students, override outcomes, excuse attendance, edit grades, or manage memberships.
5. For profile drill-down, call `GET /academic-oversight/students/:studentMemberId` with the returned `student.id`.

6. For standing drill-down, call `GET /academic-oversight/students/:studentMemberId/standing`.

Backend flow:

1. `authJwt`, `requireEmailVerified`, and `institutionAccess()` validate authenticated tenant access.
2. Service requires an active actor membership. Owner/admin gets institution-wide read scope.
3. Non-admin actors must have an active Head assignment with the required academic permission from the backend catalog.
4. Service derives student membership ids from tenant DB records and applies any query filters as narrowing constraints.
5. Risk routes compute read-only risk rows and return compact safe data only.
6. Profile and standing details are gated by their own scoped permissions and response shapes.

Academic Decisions

AcademicDecision is an auditable workflow for enrollment-drop recommendations. It does not let Heads directly drop students. All routes use `authJwt`, `requireEmailVerified`, and `institutionAccess()`. The service performs all role, ownership, Head scope, tenant, state-transition, and current-record checks.

Supported V1 decision type:

- `enrollment_drop`

Supported statuses:

- `recommended`
- `approved`
- `applying`
- `rejected`
- `applied`
- `cancelled`

Open states are `recommended`, `approved`, and `applying`. Closed states are `rejected`, `applied`, and `cancelled`.

Allowed transitions:

- Head: `recommended` create
- Head: `recommended -> cancelled` for their own recommendation only
- Admin/Owner: `recommended -> approved`
- Admin/Owner: `recommended -> rejected`
- Admin/Owner: `approved -> applying -> applied`

Method	Path	Request / query	Success	Authority / notes
POST	`/institutions/:institutionId/academic-decisions/enrollment-drop`	JSON `enrollmentId`, `studentMemberId`, `reason`.	`201 { academicDecision }`	Head only through `academicDecision.recommend_scope`; target Enrollment must be exact, active/enrolled, same tenant, same student, and inside current Head scope. Does not update Enrollment.
GET	`/institutions/:institutionId/academic-decisions`	Query `status`, `decisionType`, `studentMemberId`, `sectionId`, `recommendedByUserId`, `page`, `limit`.	`200 { academicDecisions, pagination }`	Head list is limited to decisions whose stored `sectionId` is inside current Head scope. Admin/Owner institution-wide read is supported for review users.
GET	`/institutions/:institutionId/academic-decisions/review-queue`	Query `status`, `decisionType`, `studentMemberId`, `sectionId`, `recommendedByUserId`, `page`, `limit`.	`200 { academicDecisions, pagination }`	Admin/Owner review queue. Defaults to `status=recommended`. Head, Coordinator, Mentor, normal instructor, and student are blocked.
GET	`/institutions/:institutionId/academic-decisions/:decisionId`	Path `decisionId`.	`200 { academicDecision }`	Tenant-scoped by `_id + institutionId`. Head can read only inside current Head section scope. Admin/Owner can read institution decisions for review.
POST	`/institutions/:institutionId/academic-decisions/:decisionId/approve`	Optional JSON `note` up to 1000 chars.	`200 { academicDecision }`	Admin/Owner only. Requires `recommended + isOpen=true + decisionType=enrollment_drop`. Self-review is blocked. Does not update Enrollment.
POST	`/institutions/:institutionId/academic-decisions/:decisionId/reject`	JSON `reason` 10..1000 chars.	`200 { academicDecision }`	Admin/Owner only. Requires `recommended + isOpen=true + decisionType=enrollment_drop`. Self-review is blocked. Rejected is terminal in V1.
POST	`/institutions/:institutionId/academic-decisions/:decisionId/apply`	No body.	`200 { academicDecision, enrollment }`	Admin/Owner only. Requires approved enrollment-drop decision. Drops exactly the linked Enrollment and then marks the decision applied.
PATCH	`/institutions/:institutionId/academic-decisions/:decisionId/cancel`	No body.	`200 { academicDecision }`	Recommending Head only while `status=recommended`. Does not delete the decision or update Enrollment.

Create recommendation request:

```
{
  "enrollmentId": "507f1f77bcf86cd799439061",
  "studentMemberId": "507f1f77bcf86cd799439071",
  "reason": "Repeated academic and attendance risk requires administrative review for this
enrollment."
}
```

Decision response shape:

```
{
  "academicDecision": {
    "id": "507f1f77bcf86cd799439081",
    "institutionId": "507f1f77bcf86cd799439011",
    "studentMemberId": "507f1f77bcf86cd799439071",
    "enrollmentId": "507f1f77bcf86cd799439061",
    "sectionId": "507f1f77bcf86cd799439051",
    "decisionType": "enrollment_drop",
    "status": "recommended",
    "isOpen": true,
    "reason": "Repeated academic and attendance risk requires administrative review for this
enrollment.",
    "evidenceSnapshot": {
      "capturedAt": "2026-06-16T13:06:56.976Z",
      "attendanceRisk": { "level": "warning" },
      "gradeRisk": { "level": "good" },
      "academicStanding": { "standing": "no_standing_yet", "dataUnavailable": false }
```

```
    },
    "scopeSnapshot": {
      "responsibilityType": "head",
      "matchedAssignmentIds": ["507f1f77bcf86cd799439091"],
      "institutionWide": false,
      "departmentIds": ["507f1f77bcf86cd799439041"],
      "classLevelIds": [],
      "sectionId": "507f1f77bcf86cd799439051"
    },
    "reviewSnapshot": null,
    "recommendedByUserId": "507f1f77bcf86cd799439023",
    "approvedByUserId": null,
    "rejectedByUserId": null,
    "applyingByUserId": null,
    "appliedByUserId": null,
    "cancelledByUserId": null,
    "recommendedAt": "2026-06-16T13:06:56.976Z",
    "approvedAt": null,
    "rejectedAt": null,
    "applyingAt": null,
    "appliedAt": null,
    "cancelledAt": null,
    "history": [
      {
        "action": "recommended",
        "fromStatus": null,
        "toStatus": "recommended",
        "actorUserId": "507f1f77bcf86cd799439023",
        "at": "2026-06-16T13:06:56.976Z",
        "note": null
      }
    ]
  }
}
```

Apply response shape:

```
{
  "academicDecision": {
    "id": "507f1f77bcf86cd799439081",
    "decisionType": "enrollment_drop",
    "status": "applied",
    "studentMemberId": "507f1f77bcf86cd799439071",
    "enrollmentId": "507f1f77bcf86cd799439061",
    "sectionId": "507f1f77bcf86cd799439051",
    "recommendedAt": "2026-06-16T13:06:56.976Z",
    "approvedAt": "2026-06-16T13:10:00.000Z",
    "appliedAt": "2026-06-16T13:15:00.000Z"
  },
  "enrollment": {
    "id": "507f1f77bcf86cd799439061",
    "sectionId": "507f1f77bcf86cd799439051",
    "studentMemberId": "507f1f77bcf86cd799439071",
```

```
    "status": "dropped"
  }
}
```

Frontend workflow:

1. Head logs in and calls `POST /academic-decisions/enrollment-drop` with the exact `enrollmentId`, `studentMemberId`, and a reason.
2. Backend creates the recommendation, builds evidence/scope snapshots, and sends `academic_decision_review_required` notifications to eligible Admin/Owner reviewers.
3. Admin/Owner lists `GET /academic-decisions/review-queue`, opens the decision detail, then approves or rejects.
4. Approval keeps the decision open and notifies the recommending Head with `academic_decision_approved`. Rejection closes the decision and notifies the Head with `academic_decision_rejected`.
5. Admin/Owner applies an approved decision through `POST /:decisionId/apply`. This is the only AcademicDecision route that mutates Enrollment.
6. Successful application changes only the exact Enrollment from `enrolled` to `dropped`, marks the decision `applied`, notifies the Head with `academic_decision_applied`, and notifies the affected student with `enrollment_status_changed`.
7. Head can read the final status if the decision's section remains inside their current Head scope.

Frontend safety notes:

- Do not send `status`, `decisionType`, `sectionId`, evidence snapshots, scope snapshots, actor ids, timestamps, or history from the client.
- Do not expose approval/apply buttons to Head-only users.
- Do not expose apply for `recommended`, `rejected`, `cancelled`, or already inconsistent decisions.
- Treat `applying` as an in-progress/recovery state. The frontend should show a disabled "application in progress" state and allow refresh/retry only for Admin/Owner.
- Duplicate create for the same open Enrollment returns `409 { "message": "An open enrollment drop recommendation already exists for this enrollment" }`; frontend should find/list the existing decision instead of retrying create forever.
- Notification messages intentionally omit private grade, GPA, attendance counts, and review notes. Use decision detail for authorized evidence display.

Backend consistency notes:

- New recommendations, approvals, rejections, cancellations, application claims, and completions append history.
- Application uses `approved -> applying -> applied` because the repo has no existing MongoDB transaction/session pattern.
- The Enrollment drop stores `droppedAt`, `droppedByUserId`, and `droppedByAcademicDecisionId` for audit/recovery linkage.

- If AcademicDecision and Enrollment state cannot be reconciled safely, apply returns `409` manual-recovery style errors rather than guessing.
- Approval/rejection/cancellation resolve all unread `academic_decision_review_required` notifications for that decision inside the institution.

Notifications

Minimal in-app notifications for AcademicDecision and enrollment-status events. Routes use `authJwt`, `requireEmailVerified`, and `institutionAccess()`. Users can only list/update their own notifications.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/notifications`	Query `unreadOnly`, `page`, `limit`.	`200 { notifications, pagination }`	Tenant-scoped by `institutionId + recipientUserId`; sorted recent first.
PATCH	`/institutions/:institutionId/notifications/:notificationId/read`	No body	`200 { notification }`	Updates only `_id + institutionId + recipientUserId`; cannot mark another user's notification read.

Notification response shape:

```
{
  "notification": {
    "id": "507f1f77bcf86cd799439101",
    "institutionId": "507f1f77bcf86cd799439011",
    "recipientUserId": "507f1f77bcf86cd799439021",
    "type": "academic_decision_review_required",
    "title": "Enrollment drop recommendation requires review",
    "message": "A Head submitted an enrollment drop recommendation that requires administrative
review.",
    "data": {
      "academicDecisionId": "507f1f77bcf86cd799439081",
      "studentMemberId": "507f1f77bcf86cd799439071",
      "enrollmentId": "507f1f77bcf86cd799439061",
      "sectionId": "507f1f77bcf86cd799439051"
    },
    "createdByUserId": "507f1f77bcf86cd799439023",
    "readAt": null,
    "createdAt": "2026-06-16T13:06:57.000Z",
    "updatedAt": "2026-06-16T13:06:57.000Z"
  }
}
```

Current notification types:

- `academic_decision_review_required`
- `academic_decision_cancelled`
- `academic_decision_approved`
- `academic_decision_rejected`
- `academic_decision_applied`
- `enrollment_status_changed`

Academic Years

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/academic-years`	JSON `name`, `startDate`, `endDate`, optional `status`.	`201 { academicYear }`	`institution.manage`; one active academic year per institution; normalized-name unique.
GET	`/institutions/:institutionId/academic-years`	Query `status`, `search` / `name`, `page`, `limit`.	`200 { academicYears, pagination }`	`institution.manage`.
GET	`/institutions/:institutionId/academic-years/active`	No body.	`200 { academicYear }`	Active institution membership; `404 Active academic year not found` if none.
GET	`/institutions/:institutionId/academic-years/:academicYearId`	Path `academicYearId`.	`200 { academicYear }`	`institution.manage`; tenant-scoped.
PATCH	`/institutions/:institutionId/academic-years/:academicYearId`	JSON patch `name`, dates, `status`.	`200 { academicYear }`	`institution.manage`; archived years cannot be updated/activated.
POST	`/institutions/:institutionId/academic-years/:academicYearId/archive`	No body.	`200 { academicYear }`	`institution.manage`; status-only archive.

Departments

University-facing module. Department routes are tenant-scoped by `institutionId`. Department list/get currently require active institution access but no route-level `department.manage`.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/departments`	JSON `name`, `code`, `description`.	`201 { department }`	`department.manage`; university-only; unique code/name.
GET	`/institutions/:institutionId/departments`	Query `status`.	`200 { total, departments }`	Institution access.
GET	`/institutions/:institutionId/departments/:departmentId`	Path `departmentId`.	`200 { department }`	Institution access.
PATCH	`/institutions/:institutionId/departments/:departmentId`	JSON patch `name`, `code`, `description`, `status`.	`200 { department }`	`department.manage`; rejects empty/no-valid-field patch.
DELETE	`/institutions/:institutionId/departments/:departmentId`	No body.	`200 { message }`	`department.manage`; TODO/VERIFY destructive cascading expectations.

Classes / Class Levels

School-only structure module. No delete routes exist for class levels, school classes, or class assignments.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/class-levels`	JSON `name`, `code`, `description`.	`201 { classLevel }`	`class.manage`; school-only; unique code/name.
GET	`/institutions/:institutionId/class-levels`	Query `status`.	`200 { total, classLevels }`	`class.manage`.
GET	`/institutions/:institutionId/class-levels/:classLevelId`	Path `classLevelId`.	`200 { classLevel }`	`class.manage`.
PATCH	`/institutions/:institutionId/class-levels/:classLevelId`	JSON patch `name`, `code`, `description`, `status`.	`200 { classLevel }`	`class.manage`.
POST	`/institutions/:institutionId/classes`	JSON `classLevelId`, `name`, `code`.	`201 { schoolClass }`	Active class level required.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/classes`	Query `status`, `classLevelId`.	`200 { total, schoolClasses }`	`status` supports `active`, `archived`, or `all`.
GET	`/institutions/:institutionId/classes/:classId`	Path `classId`.	`200 { schoolClass }`	Tenant-scoped.
PATCH	`/institutions/:institutionId/classes/:classId`	JSON patch `name`, `code`, `status`.	`200 { schoolClass }`	`classLevelId` is not changed after create.
POST	`/institutions/:institutionId/classes/:classId/students`	JSON `semesterId`, `studentMemberId`.	`201 { assignment }`	Creates/reactivates assignment and syncs matching school section enrollments.
GET	`/institutions/:institutionId/classes/:classId/students`	Query `semesterId`, `status`.	`200 { assignments }`	Default status is `assigned`.
GET	`/institutions/:institutionId/classes/:classId/students/:assignmentId`	Path `assignmentId`.	`200 { assignment }`	Tenant/class scoped.
PATCH	`/institutions/:institutionId/classes/:classId/students/:assignmentId/status`	JSON `status` (`assigned` or `removed`).	`200 { assignment }`	Removal marks matching school enrollments dropped.

Subjects

`Subject` is shared. University subjects are scoped through `departmentId`; school subjects are scoped through `classLevelId`.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/departments/:departmentId/subjects`	JSON `name`, `code`, `description`, `creditHours`, `prerequisiteSubjectIds`, `status`.	`201 { subject }`	University subject; `creditHours` required; prerequisites tenant-scoped and deduped.
POST	`/institutions/:institutionId/class-levels/:classLevelId/subjects`	JSON `name`, `code`, `description`, `status`.	`201 { subject }`	School subject; rejects `creditHours` and department scope.
GET	`/institutions/:institutionId/subjects`	Query `departmentId`, `classLevelId`, `active`.	`200 { total, subjects }`	`subject.manage`.
GET	`/institutions/:institutionId/subjects/:subjectId`	Path `subjectId`.	`200 { subject }`	Tenant-scoped.
PATCH	`/institutions/:institutionId/subjects/:subjectId`	JSON patch subject fields.	`200 { subject }`	Institution type controls allowed fields.
DELETE	`/institutions/:institutionId/subjects/:subjectId`	No body.	`200 { message }`	TODO/VERIFY linked section/assessment delete guards.

Memberships / Users

`User` is auth identity only. `Membership` is institution access + role. `MemberProfile` is personal/member info.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/members`	JSON or multipart: `email`, `role`, `profile`; optional `profilePicture`.	`201 { member }`	`membership.manage`; supports text/plain JSON and multipart `payload` / `profile` JSON strings.
GET	`/institutions/:institutionId/members`	Query `role`, `status`, `search`, `page`, `limit`.	`200 { total, members }`	List response is summary shape.
GET	`/institutions/:institutionId/members/me`	No body.	`200 { member }`	Self-only full member/profile read. Requires active institution membership, not `membership.manage`; missing `MemberProfile` returns `profile: null`.
GET	`/institutions/:institutionId/members/:membershipId`	Path `membershipId`.	`200 { member }`	Detail response includes profile details.

Method	Path	Request / query	Success	Notes
PATCH	`/institutions/:institutionId/members/:membershipId`	JSON or multipart profile patch; optional `profilePicture`.	`200 { member }`	Uploaded file cleaned up when service rejects.
PATCH	`/institutions/:institutionId/members/:membershipId/status`	JSON `status` (`active` or `suspended`).	`200 { member }`	Does not change role.

`GET /institutions/:institutionId/members/me` uses the authenticated active membership from `institutionAccess()` and re-reads it by `_id + institutionId + userId + status=active`. It never accepts a client-supplied membership id. `MemberProfile` is loaded by `institutionId + membershipId`, and the route is registered before `/members/:membershipId` so the literal `me` is not treated as a membership id. The existing admin detail route remains protected by `membership.manage`.

Sections

Section list/detail uses route-level `section.read` plus service-level role-aware scoping.

Method	Path	Request / query	Success	Auth / scoping notes
POST	`/institutions/:institutionId/sections`	JSON `semesterId`, `subjectId`, `schoolClassId`, `code`, `title`, `instructorMemberId`, `capacity`, `schedule`, `status`.	`201 { section }`	`section.manage`; validates semester teaching days and room conflicts; school sections require `schoolClassId` and backfill class roster enrollments.
GET	`/institutions/:institutionId/sections`	Query `semesterId`, `subjectId`, `instructorMemberId`, `departmentId`, `schoolClassId`, `status`.	`200 { sections }`	Owner/admin broad; instructors only own sections; coordinator via `section.read_scope`; students blocked.
GET	`/institutions/:institutionId/sections/:sectionId`	Path `sectionId`.	`200 { section }`	Same read scoping as list; scoped readers get safer detail without creator audit metadata.
PATCH	`/institutions/:institutionId/sections/:sectionId`	JSON patch section fields.	`200 { section }`	`section.manage`; room conflict and schedule validation apply.
DELETE	`/institutions/:institutionId/sections/:sectionId`	No body.	`200 { message }`	TODO/VERIFY delete guards for linked enrollment/attendance data.

Section Sessions

Routes use `institutionAccess()` and service-level authorization. Allowed access includes admin/owner, parent section instructor, and selected derived coordinator responsibility scopes.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/sessions`	JSON `sessionType`, `sessionDate`, `startTime`, `endTime`, `room`.	`201 { sectionSession }`	`sessionType` supports `lecture` or `lab`; date must be inside semester and teaching days.
GET	`/institutions/:institutionId/sections/:sectionId/sessions`	Query `fromDate`, `toDate`, `status`, `attendanceState`, `page`, `limit`.	`200 { items, pagination }`	Uses coordinator `sectionSession.read_scope` for scoped responsibility access.
POST	`/institutions/:institutionId/sections/:sectionId/sessions/generate`	Optional JSON `fromDate`, `toDate`.	`200 { message, generatedCount, skippedCount, sectionSessions }`	Generates from recurring section schedule; skips off/exam dates and existing sessions.
GET	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId`	Path `sectionSessionId`.	`200 { sectionSession }`	Includes lightweight `section` and `subject`.
PATCH	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/status`	JSON `status`, optional `note`.	`200 { sectionSession }`	Dedicated status endpoint; cancellation emails are best-effort to active enrolled students.

Method	Path	Request / query	Success	Notes
PATCH	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId`	JSON patch non-status fields.	`200 { sectionSession }`	Rejects status metadata updates.

Attendance

Attendance is anchored to `SectionSession`.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/open`	No body.	`200 { sectionSession }`	Opens attendance; cancelled/closed sessions rejected.
POST	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/qr-token`	No body.	`200 { qrToken, sectionSession }`	Token expires after 20 seconds; token call replaces prior token.
PUT	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/manual`	JSON `entries[]` with `enrollmentId`, `attendanceStatus`, optional `note`.	`200 { attendanceRecords }`	Status must be `present`, `absent`, `late`, or `excused`; duplicate enrollment ids rejected.
GET	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/roster`	Query `filter`.	`200 { attendanceRoster }`	Filters: `all`, `attended`, `not_attended`, `present`, `absent`, `late`, `excused`, `pending`.
POST	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/close`	No body.	`200 { sectionSession }`	Marks missing active enrollments absent.
POST	`/institutions/:institutionId/sections/:sectionId/sessions/:sectionSessionId/attendance/check-in`	JSON `token`.	`200 { attendanceRecord }`	Student-only; active enrollment required; same-day QR only; present/late computed by grace window.
GET	`/institutions/:institutionId/sections/:sectionId/attendance/analytics`	Query `fromDate`, `toDate`.	`200 { attendanceAnalytics }`	Closed non-cancelled sessions only; attendance risk is response-only and does not change enrollments/outcomes.
GET	`/institutions/:institutionId/students/me/attendance-history`	Query `sectionId`, `fromDate`, `toDate`, `status`.	`200 { attendanceHistory, summary }`	Student self route.

Enrollments / Self-Enrollment

Shared `Enrollment` model; school enrollments are backend-managed through class roster sync.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/enrollments`	JSON `studentMemberId`.	`201 { enrollment }`	`section.manage`; blocked for school sections.
GET	`/institutions/:institutionId/sections/:sectionId/enrollments`	No query.	`200 { enrollments }`	Section-scoped.
GET	`/institutions/:institutionId/sections/:sectionId/enrollments/:enrollmentId`	Path `enrollmentId`.	`200 { enrollment }`	Tenant + section scoped.
PATCH	`/institutions/:institutionId/sections/:sectionId/enrollments/:enrollmentId/status`	JSON `status` (`enrolled` or `dropped`).	`200 { enrollment }`	Use status update to re-enroll.
GET	`/institutions/:institutionId/students/me/self-enrollment/sections`	Query `availableOnly`.	`200 { sections }`	Student-only, university-only active-semester browse.
GET	`/institutions/:institutionId/students/me/self-enrollment/sections/:sectionId/eligibility`	No body.	`200 { eligibility }`	Preview route with blocking reason codes.
POST	`/institutions/:institutionId/students/me/self-enrollment/submissions`	JSON `sectionIds[]`.	`201 { enrollments, submission }`	Validates whole selected set before writes; duplicate same-subject and min/max load rules.

Assessments / Results

Assessment routes use service-level section-derived authorization. Student self routes expose only current student data.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/assessments`	JSON `title`, `description`, `type`, `assessmentDate` or `sectionSessionId`, `maxScore`, `weight`, `status`.	`201 { assessment }`	Snapshots active enrollments into result rows.
GET	`/institutions/:institutionId/sections/:sectionId/assessments`	Query `status`.	`200 { assessments }`	Staff/scoped read.
GET	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId`	Path `assessmentId`.	`200 { assessment }`	Tenant + section scoped.
PATCH	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId`	JSON patch allowed fields.	`200 { assessment }`	Archived read-only; only draft assessments can be updated.
PATCH	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId/status`	JSON `status`.	`200 { assessment }`	Status supports `draft`, `announced`, `published`, or `archived`; emails best-effort on announced/published.
PUT	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId/results`	JSON `entries[]` with `enrollmentId`, `score?`, `note?`.	`200 { assessmentResults }`	Score numeric, `0..maxScore`; unique enrollment ids.
GET	`/institutions/:institutionId/sections/:sectionId/assessments/:assessmentId/results`	No body.	`200 { assessmentResults }`	Staff/scoped read.
GET	`/institutions/:institutionId/sections/:sectionId/results`	No body.	`200 { sectionResults }`	Published-only weighted section results.
GET	`/institutions/:institutionId/students/me/assessments`	No body.	`200 { assessments }`	Student sees announced/published metadata; no draft/archived.
GET	`/institutions/:institutionId/students/me/results`	Query `sectionId`, `semesterId`.	`200 { results }`	Student sees published results only.

Homework / Projects

Homework is separate from AssessmentResult. Optional `assessmentId` links graded work but homework submissions do not become scores.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/homeworks`	JSON `title`, `description`, `type`, `dueAt`, `sectionSessionId`, `assessmentId`, `submissionMode`, `allowLate`, `status`.	`201 { homework }`	Assignment emails best-effort when created as assigned.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks`	Query `status`, `type`, `submissionMode`, `assessmentId`, `page`, `limit`.	`200 { homeworks, pagination }`	Staff/scoped section access.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId`	Path `homeworkId`.	`200 { homework }`	Staff/scoped section access.
PATCH	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId`	JSON patch homework fields.	`200 { homework }`	`submissionMode` cannot change after submissions exist.
PATCH	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/status`	JSON `status`.	`200 { homework }`	Emails best-effort on `draft -> assigned`.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submissions`	No body.	`200 { homework, submissions }`	Roster combines active enrollments with existing submissions.

Method	Path	Request / query	Success	Notes
PUT	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/enrollments/:enrollmentId/submission`	JSON `status`, `instructorFeedback`.	`200 { submission }`	Offline/manual mode only.
PATCH	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submissions/:submissionId`	JSON `status`, `instructorFeedback`.	`200 { submission }`	Staff submission update.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submissions/:submissionId/file`	No body.	File download	Private file authorization before `res.download`.
GET	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submission`	No body.	`200 { homework, submission }`	Student self route.
POST	`/institutions/:institutionId/sections/:sectionId/homeworks/:homeworkId/submission`	multipart field `submissionFile`, optional `studentNote`.	`201 { submission }`	Student-only; requires assigned `online_upload`; late blocked unless allowed.
GET	`/institutions/:institutionId/students/me/homeworks`	Query `status`, `type`, `dueFrom`, `dueTo`, `sectionId`, `page`, `limit`.	`200 { homeworks, pagination }`	Student sees assigned/closed items for active enrollments.

Section Materials

PDF-only private section resources, separate from Homework.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/sections/:sectionId/materials`	multipart `materialFile`, plus `title`, `description`, `materialType`, `sectionSessionId`, `status`.	`201 { material }`	PDF required; private storage.
GET	`/institutions/:institutionId/sections/:sectionId/materials`	Query `status`, `materialType`, `page`, `limit`.	`200 { materials, pagination }`	Staff/scoped section access.
GET	`/institutions/:institutionId/sections/:sectionId/materials/:materialId`	Path `materialId`.	`200 { material }`	Staff/scoped detail.
PATCH	`/institutions/:institutionId/sections/:sectionId/materials/:materialId`	multipart optional `materialFile` plus patch fields.	`200 { material }`	Replacing file removes old private file only after DB update.
PATCH	`/institutions/:institutionId/sections/:sectionId/materials/:materialId/status`	JSON `status`.	`200 { material }`	Status supports `draft`, `published`, or `archived`; archived restore deferred.
GET	`/institutions/:institutionId/sections/:sectionId/materials/:materialId/file`	No body.	File download	Student can download only published material for active enrolled section.
GET	`/institutions/:institutionId/students/me/materials`	Query `sectionId`, `materialType`, `page`, `limit`.	`200 { materials, pagination }`	Student self list, published only.

Academic Profiles / Policies / Standing

Academic profile facade delegates by `Institution.type`.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/students/me/academic-profile`	Query `academicYearId`, `semesterId`.	`200 { academicProfile }`	Student self only; school currently requires `academicYearId`.
GET	`/institutions/:institutionId/students/:studentMembershipId/academic-profile`	Query `academicYearId`, `semesterId`.	`200 { academicProfile }`	Route uses `membership.manage`; providers also enforce school `class.manage` / university `section.manage` style access.
POST	`/institutions/:institutionId/academic-policies`	JSON university policy config.	`201 { academicPolicy }`	University-only; semester-effective.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/academic-policies`	Query `status`.	`200 { academicPolicies }`	`status` supports `draft`, `active`, `archived`, or `all`.
GET	`/institutions/:institutionId/academic-policies/:policyId`	Path `policyId`.	`200 { academicPolicy }`	Tenant-scoped.
PATCH	`/institutions/:institutionId/academic-policies/:policyId`	JSON patch university policy config.	`200 { academicPolicy }`	Validates repeatPolicy and academicStandingRules.
GET	`/institutions/:institutionId/academic-standing/university/summary`	Query `standing`, `minGpa`, `maxGpa`, `semesterId`, `departmentId`, `search`, `page`, `limit`.	`200 { standingSummary }`	Admin/owner institution-wide; guarded by `section.manage`; university-only.
GET	`/institutions/:institutionId/academic-standing/university/students`	Same filters.	`200 { standingSummary, students, pagination }`	Department filtering derived through Subject.
GET	`/institutions/:institutionId/instructors/me/academic-standing/summary`	Same filters.	`200 { standingSummary }`	Instructor-scoped to assigned sections and active enrolled students.
GET	`/institutions/:institutionId/instructors/me/academic-standing/students`	Same filters.	`200 { standingSummary, students, pagination }`	Standing is live/read-only, not stored.

School Academic Policies / Report Cards / Final Outcomes

School policy and outcomes are percentage/promotion based and stay separate from university GPA/credit-hour behavior.

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/academic-policies/school`	JSON school policy config.	`201 { schoolAcademicPolicy }`	School-only; requires real non-archived `appliesTo.academicYearId`.
GET	`/institutions/:institutionId/academic-policies/school`	Query `status`, `academicYearId`, `schoolStageId`, `classLevelId`.	`200 { schoolAcademicPolicies }`	Resolver supports academic-year/stage/class-level fallback.
GET	`/institutions/:institutionId/academic-policies/school/:policyId`	Path `policyId`.	`200 { schoolAcademicPolicy }`	Tenant-scoped.
PATCH	`/institutions/:institutionId/academic-policies/school/:policyId`	JSON patch school policy config.	`200 { schoolAcademicPolicy }`	Boolean fields must be booleans; semester weights total 100.
POST	`/institutions/:institutionId/academic-policies/school/:policyId/archive`	No body.	`200 { schoolAcademicPolicy }`	Status-only lifecycle.
GET	`/institutions/:institutionId/school-outcomes/students/:studentMembershipId/semesters/:semesterId/report-card`	No body.	`200 { reportCard }`	`class.manage`; read-only.
GET	`/institutions/:institutionId/school-outcomes/students/:studentMembershipId/academic-years/:academicYearId/report-card`	No body.	`200 { reportCard }`	Includes live promotion decision explanation.
GET	`/institutions/:institutionId/school-outcomes/classes/:schoolClassId/academic-years/:academicYearId/promotion-summary`	No body.	`200 { promotionSummary }`	Current MVP unpaginated.
GET	`/institutions/:institutionId/school-outcomes/academic-years/:academicYearId/promotion-dashboard/summary`	Query admin dashboard filters.	`200 { promotionSummary }`	Uses final outcomes when present, live calculation otherwise.
GET	`/institutions/:institutionId/school-outcomes/academic-years/:academicYearId/promotion-dashboard/students`	Query `decision`, `schoolClassId`, `classLevelId`, `complete`, `hasMissingResults`, `finalized`, `search`, `page`, `limit`.	`200 { promotionSummary, students, pagination }`	Admin/staff full promotion dashboard.
GET	`/institutions/:institutionId/instructors/me/school-outcomes/academic-years/:academicYearId/promotion-dashboard/summary`	Query instructor-safe filters.	`200 { sectionOutcomeSummary }`	Instructor assigned-section only; rejects `decision` and `finalized`.

Method	Path	Request / query	Success	Notes
GET	`/institutions/:institutionId/instructors/me/school-outcomes/academic-years/:academicYearId/promotion-dashboard/students`	Query `schoolClassId`, `classLevelId`, `complete`, `hasMissingResults`, `search`, `page`, `limit`.	`200 { sectionOutcomeSummary, sectionOutcomes, pagination }`	No full yearly decision exposed.
POST	`/institutions/:institutionId/school-outcomes/students/:studentMembershipId/academic-years/:academicYearId/finalize`	Optional JSON `override.finalDecision`, `override.reason`.	`201 { schoolFinalOutcome }`	Does not move student classes.
PATCH	`/institutions/:institutionId/school-outcomes/final-outcomes/:outcomeId/override`	JSON `finalDecision`, `reason` or `override` object.	`200 { schoolFinalOutcome }`	Cancelled outcomes cannot be overridden.
GET	`/institutions/:institutionId/school-outcomes/students/:studentMembershipId/academic-years/:academicYearId/final-outcome`	No body.	`200 { schoolFinalOutcome }`	`404` if no final outcome exists.

Semesters

Method	Path	Request / query	Success	Notes
POST	`/institutions/:institutionId/semesters`	JSON `academicYearId?`, `name`, `startDate`, `endDate`, `teachingDays`, `weeksCount`, `calendarOverrides`, `examDates`, `status`.	`201 { semester }`	`semester.manage`; one active semester per institution.
GET	`/institutions/:institutionId/semesters`	Query `status`.	`200 { total, semesters }`	`semester.manage`.
GET	`/institutions/:institutionId/semesters/active`	No body.	`200 { semester }`	Institution access only.
GET	`/institutions/:institutionId/semesters/:semesterId`	Path `semesterId`.	`200 { semester }`	Detail includes weeks.
PATCH	`/institutions/:institutionId/semesters/:semesterId`	JSON patch allowed semester fields.	`200 { semester }`	Optional `academicYearId` must reference same-tenant non-archived year.
DELETE	`/institutions/:institutionId/semesters/:semesterId`	No body.	`200 { message }`	TODO/VERIFY linked data delete guards.

Examples By Module

Auth

```
POST /auth/login
Content-Type: application/json

{ "email": "admin@example.test", "password": "Password123" }
```

```
{
  "accessToken": "<JWT_TOKEN>",
  "user": {
    "id": "507f1f77bcf86cd799439011",
    "email": "admin@example.test",
    "status": "active",
    "isEmailVerified": true
```

```
  }
}
```

Institution

```
GET /institutions/me
Authorization: Bearer <JWT_TOKEN>
```


```
{
  "user": {
    "id": "507f1f77bcf86cd799439011",
    "email": "instructor@example.test",
    "status": "active",
    "isEmailVerified": true
  },
  "institutions": [
    {
      "institutionId": "507f1f77bcf86cd799439021",
      "membershipId": "507f1f77bcf86cd799439031",
      "name": "Example University",
      "type": "university",
      "institutionStatus": "active",
      "role": "instructor",
      "membershipStatus": "active",
      "joinedAt": "2026-06-16T12:57:22.122Z",
      "isOwner": false,
      "memberProfile": {
        "fullName": "Instructor Example",
        "firstName": "Instructor",
        "lastName": "Example",
        "mobile": "+963900000001",
        "profilePictureUrl": "/uploads/member-profiles/instructor.png"
      },
      "permissions": [
        "section.read",
        "attendance.take",
        "attendance.excuse"
      ],
      "scopedCapabilities": [
        {
          "responsibilityType": "head",
          "scopeType": "institution",
          "scopeId": "507f1f77bcf86cd799439021",
          "permissions": [
            "academicRisk.read_scope",
            "academicProfile.read_scope",
            "academicStanding.read_scope",
            "academicDecision.read_scope",
            "academicDecision.recommend_scope"
          ]
        }
      ],
```

```
      "activeSemester": {
        "id": "507f1f77bcf86cd799439041",
        "academicYearId": "507f1f77bcf86cd799439051",
        "name": "Semester 1 - 2026",
        "startDate": "2026-03-12",
        "endDate": "2026-09-22",
        "status": "active"
      },
      "activeAcademicYear": {
        "id": "507f1f77bcf86cd799439051",
        "name": "2026",
        "startDate": "2026-01-01",
        "endDate": "2026-12-31",
        "status": "active"
      }
    }
  ],
  "activeInstitutionId": "507f1f77bcf86cd799439021",
  "onboarding": {
    "isEmailVerified": true,
    "hasInstitutions": true,
    "hasActiveMembership": true,
    "hasAdminMembership": false,
    "activeInstitutionType": "university",
    "nextStep": null
  }
}
```

Billing

```
POST /institutions/507f1f77bcf86cd799439011/billing/checkout-session
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{ "plan": "plus" }
```

```
{
  "checkoutSession": {
    "id": "cs_test_placeholder",
    "url": "https://checkout.stripe.com/c/pay/cs_test_placeholder"
  },
  "billingProfile": {
    "institutionId": "507f1f77bcf86cd799439011",
    "stripeCustomerId": "cus_test_placeholder",
    "stripeSubscriptionId": null,
    "stripePriceId": "price_test_placeholder",
    "plan": null,
    "billingInterval": null,
    "status": null,
    "cancelAtPeriodEnd": false,
    "trialEnd": null
  },
```

```
  "entitlements": {
    "plan": null,
    "billingInterval": null,
    "subscriptionStatus": null,
    "hasPaidAccess": false,
    "maxActiveStudents": 0,
    "maxActiveStaff": 0,
    "currentPeriodStart": null,
    "currentPeriodEnd": null,
    "cancelAtPeriodEnd": false,
    "trialEnd": null
  }
}
```

```
GET /institutions/507f1f77bcf86cd799439011/billing/status
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "billingProfile": {
    "institutionId": "507f1f77bcf86cd799439011",
    "stripeCustomerId": "cus_test_placeholder",
    "stripeSubscriptionId": "sub_test_placeholder",
    "stripePriceId": "price_plus_6_month_placeholder",
    "plan": "plus",
    "billingInterval": "six_month",
    "stripeProductId": "prod_test_placeholder",
    "status": "active",
    "currentPeriodStart": "2026-01-01T00:00:00.000Z",
    "currentPeriodEnd": "2026-07-01T00:00:00.000Z",
    "cancelAtPeriodEnd": false,
    "trialEnd": null
  },
  "entitlements": {
    "plan": "plus",
    "billingInterval": "six_month",
    "subscriptionStatus": "active",
    "hasPaidAccess": true,
    "maxActiveStudents": 300,
    "maxActiveStaff": 30,
    "currentPeriodStart": "2026-01-01T00:00:00.000Z",
    "currentPeriodEnd": "2026-07-01T00:00:00.000Z",
    "cancelAtPeriodEnd": false,
    "trialEnd": null
  }
}
```

```
POST /institutions/507f1f77bcf86cd799439011/billing/upgrade-to-pro
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "upgrade": {
```

```
    "fromPlan": "plus",
    "toPlan": "pro",
    "status": "processing"
  }
}
```

If Stripe requires customer action for the prorated invoice, the response may include only the safe Stripe.js client secret:

```
{
  "upgrade": {
    "fromPlan": "plus",
    "toPlan": "pro",
    "status": "payment_action_required",
    "clientSecret": "pi_client_secret_placeholder"
  }
}
```

The local billing profile remains Plus until a verified Stripe webhook confirms the subscription now uses the trusted Pro Price.

Responsibility Assignments

```
POST /institutions/507f1f77bcf86cd799439011/responsibility-assignments
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{
  "userId": "507f1f77bcf86cd799439023",
  "responsibilityType": "head",
  "scopeType": "department",
  "scopeId": "507f1f77bcf86cd799439041",
  "startAt": "2026-09-01T00:00:00.000Z",
  "isActive": true
}
```

```
{
  "responsibilityAssignment": {
    "id": "507f1f77bcf86cd799439051",
    "institutionId": "507f1f77bcf86cd799439011",
    "userId": "507f1f77bcf86cd799439023",
    "responsibilityType": "head",
    "scopeType": "department",
    "scopeId": "507f1f77bcf86cd799439041",
    "isActive": true,
    "startAt": "2026-09-01T00:00:00.000Z",
    "endAt": null
  }
}
```

```
GET /institutions/507f1f77bcf86cd799439011/responsibility-assignments?
responsibilityType=head&activeNow=true
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "responsibilityAssignments": [
    {
      "id": "507f1f77bcf86cd799439051",
      "userId": "507f1f77bcf86cd799439023",
      "responsibilityType": "head",
      "scopeType": "department",
      "scopeId": "507f1f77bcf86cd799439041",
      "isActive": true
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

Academic Year

```
POST /institutions/507f1f77bcf86cd799439011/academic-years
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{ "name": "2026-2027", "startDate": "2026-09-01", "endDate": "2027-06-30", "status": "active" }
```

```
{ "academicYear": { "id": "507f1f77bcf86cd799439011", "name": "2026-2027", "status": "active" } }
```

Members

```
POST /institutions/507f1f77bcf86cd799439011/members
Authorization: Bearer <JWT_TOKEN>
Content-Type: multipart/form-data

payload={"email":"student@example.test","role":"student","profile":
{"firstName":"Sara","lastName":"Ali","nationalNumber":"S-100"}}
profilePicture=@avatar.png
```

```
{ "member": { "id": "507f1f77bcf86cd799439011", "role": "student", "status": "active" } }
```

Sections

```
POST /institutions/507f1f77bcf86cd799439011/sections
Authorization: Bearer <JWT_TOKEN>
```

```
Content-Type: application/json

{
  "semesterId": "507f1f77bcf86cd799439011",
  "subjectId": "507f1f77bcf86cd799439011",
  "code": "MATH-7A-1",
  "title": "Math Grade 7 A",
  "capacity": 30,
  "schedule": [
    { "day": "monday", "startTime": "09:00", "endTime": "10:30", "room": "A101", "sessionType":
"lecture" }
  ],
  "status": "active"
}
```

```
{ "section": { "id": "507f1f77bcf86cd799439011", "code": "MATH-7A-1", "status": "active" } }
```

Attendance

```
PUT
/institutions/507f1f77bcf86cd799439011/sections/507f1f77bcf86cd799439011/sessions/507f1f77bcf86cd799
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{ "entries": [{ "enrollmentId": "507f1f77bcf86cd799439011", "attendanceStatus": "present" }] }
```

```
{ "attendanceRecords": [{ "id": "507f1f77bcf86cd799439011", "attendanceStatus": "present",
"source": "manual" }] }
```

Assessments / Results

```
PUT
/institutions/507f1f77bcf86cd799439011/sections/507f1f77bcf86cd799439011/assessments/507f1f77bcf86cd
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json

{ "entries": [{ "enrollmentId": "507f1f77bcf86cd799439011", "score": 87, "note": "Checked" }] }
```

```
{ "assessmentResults": [{ "enrollmentId": "507f1f77bcf86cd799439011", "score": 87, "note":
"Checked" }] }
```

Homework

```
POST
/institutions/507f1f77bcf86cd799439011/sections/507f1f77bcf86cd799439011/homeworks/507f1f77bcf86cd79
Authorization: Bearer <JWT_TOKEN>
Content-Type: multipart/form-data

studentNote=Completed
submissionFile=@solution.pdf
```


```
{ "submission": { "id": "507f1f77bcf86cd799439011", "status": "submitted", "originalFilename":
"solution.pdf" } }
```


Section Materials

```
POST /institutions/507f1f77bcf86cd799439011/sections/507f1f77bcf86cd799439011/materials
Authorization: Bearer <JWT_TOKEN>
Content-Type: multipart/form-data

title=Week 1 Notes
materialType=lecture_pdf
status=published
materialFile=@week-1.pdf
```


```
{ "material": { "id": "507f1f77bcf86cd799439011", "title": "Week 1 Notes", "status": "published"
} }
```


Academic Oversight

```
GET /institutions/507f1f77bcf86cd799439011/academic-oversight/students?
riskLevel=high&page=1&limit=20
Authorization: Bearer <JWT_TOKEN>
```


```
{
  "academicRiskSummary": {
    "totalStudents": 1,
    "riskLevels": { "good": 0, "warning": 0, "high": 1, "unknown": 0 },
    "attendanceRisk": { "good": 0, "warning": 0, "high": 1, "unknown": 0 },
    "gradeRisk": { "good": 1, "warning": 0, "high": 0, "unknown": 0 },
    "standingSummary": {}
  },
  "students": [
    {
      "student": {
        "id": "507f1f77bcf86cd799439071",
```

```
        "userId": "507f1f77bcf86cd799439091",
        "name": "Sara Ali",
        "email": "sara@example.test"
      },
      "riskLevel": "high",
      "attendanceRisk": {
        "level": "high",
        "absences": 6,
        "lateCount": 0,
        "excusedCount": 0,
        "effectiveAbsences": 6,
        "reasons": ["attendance_drop_risk_threshold_reached"]
      },
      "gradeRisk": {
        "level": "good",
        "currentAverage": 82,
        "missingResults": 0,
        "failedAssessments": 0,
        "reasons": []
      },
      "reasons": ["attendance_drop_risk_threshold_reached"]
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

```
GET /institutions/507f1f77bcf86cd799439011/academic-oversight/students/507f1f77bcf86cd799439071
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "student": {
    "student": {
      "id": "507f1f77bcf86cd799439071",
      "userId": "507f1f77bcf86cd799439091",
      "name": "Sara Ali",
      "email": "sara@example.test"
    },
    "riskLevel": "unknown",
    "attendanceRisk": { "level": "unknown" },
    "gradeRisk": { "level": "unknown" },
    "academicProfile": {
      "program": null,
      "department": null,
      "classLevel": null,
      "gpa": 2.4,
      "credits": { "earned": 30, "attempted": 36 },
      "status": "active",
      "dataUnavailable": false
    },
    "reasons": []
  }
}
```

```
GET /institutions/507f1f77bcf86cd799439011/academic-
oversight/students/507f1f77bcf86cd799439071/standing
Authorization: Bearer <JWT_TOKEN>
```


```
{
  "student": {
    "student": {
      "id": "507f1f77bcf86cd799439071",
      "userId": "507f1f77bcf86cd799439091",
      "name": "Sara Ali",
      "email": "sara@example.test"
    },
    "riskLevel": "warning",
    "academicStanding": {
      "standing": "probation",
      "requiresReview": false,
      "reason": "gpa_in_probation_band",
      "dataUnavailable": false
    },
    "reasons": ["gpa_in_probation_band"]
  }
}
```


Academic Decisions

Head recommends one exact Enrollment drop:

```
POST /institutions/507f1f77bcf86cd799439011/academic-decisions/enrollment-drop
Authorization: Bearer <HEAD_JWT_TOKEN>
Content-Type: application/json

{
  "enrollmentId": "507f1f77bcf86cd799439061",
  "studentMemberId": "507f1f77bcf86cd799439071",
  "reason": "Repeated academic and attendance risk requires administrative review for this
enrollment."
}
```


```
{
  "academicDecision": {
    "id": "507f1f77bcf86cd799439081",
    "decisionType": "enrollment_drop",
    "status": "recommended",
    "isOpen": true,
    "studentMemberId": "507f1f77bcf86cd799439071",
    "enrollmentId": "507f1f77bcf86cd799439061",
    "sectionId": "507f1f77bcf86cd799439051",
    "recommendedByUserId": "507f1f77bcf86cd799439023",
    "evidenceSnapshot": {
      "attendanceRisk": { "level": "warning" },
```

```
      "gradeRisk": { "level": "good" },
      "academicStanding": { "standing": "no_standing_yet" }
    },
    "scopeSnapshot": {
      "responsibilityType": "head",
      "institutionWide": false,
      "sectionId": "507f1f77bcf86cd799439051"
    }
  }
}
```

Admin lists the review queue:

```
GET /institutions/507f1f77bcf86cd799439011/academic-decisions/review-queue?limit=20
Authorization: Bearer <ADMIN_JWT_TOKEN>
```

```
{
  "academicDecisions": [
    {
      "id": "507f1f77bcf86cd799439081",
      "decisionType": "enrollment_drop",
      "status": "recommended",
      "isOpen": true,
      "studentMemberId": "507f1f77bcf86cd799439071",
      "enrollmentId": "507f1f77bcf86cd799439061",
      "sectionId": "507f1f77bcf86cd799439051"
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

Admin approves:

```
POST /institutions/507f1f77bcf86cd799439011/academic-decisions/507f1f77bcf86cd799439081/approve
Authorization: Bearer <ADMIN_JWT_TOKEN>
Content-Type: application/json

{
  "note": "Reviewed current records and approved for application."
}
```

```
{
  "academicDecision": {
    "id": "507f1f77bcf86cd799439081",
    "decisionType": "enrollment_drop",
    "status": "approved",
    "isOpen": true,
    "approvedByUserId": "507f1f77bcf86cd799439021",
    "approvedAt": "2026-06-16T13:10:00.000Z",
    "reviewSnapshot": {
```

```
      "reviewedAt": "2026-06-16T13:10:00.000Z",
      "enrollmentStatus": "enrolled",
      "evidenceChanged": false
    }
  }
}
```

Admin applies:

```
POST /institutions/507f1f77bcf86cd799439011/academic-decisions/507f1f77bcf86cd799439081/apply
Authorization: Bearer <ADMIN_JWT_TOKEN>
```

```
{
  "academicDecision": {
    "id": "507f1f77bcf86cd799439081",
    "decisionType": "enrollment_drop",
    "status": "applied",
    "studentMemberId": "507f1f77bcf86cd799439071",
    "enrollmentId": "507f1f77bcf86cd799439061",
    "sectionId": "507f1f77bcf86cd799439051",
    "recommendedAt": "2026-06-16T13:06:56.976Z",
    "approvedAt": "2026-06-16T13:10:00.000Z",
    "appliedAt": "2026-06-16T13:15:00.000Z"
  },
  "enrollment": {
    "id": "507f1f77bcf86cd799439061",
    "sectionId": "507f1f77bcf86cd799439051",
    "studentMemberId": "507f1f77bcf86cd799439071",
    "status": "dropped"
  }
}
```

Head checks notifications:

```
GET /institutions/507f1f77bcf86cd799439011/notifications?unreadOnly=true&limit=20
Authorization: Bearer <HEAD_JWT_TOKEN>
```

```
{
  "notifications": [
    {
      "id": "507f1f77bcf86cd799439101",
      "type": "academic_decision_approved",
      "title": "Enrollment drop recommendation approved",
      "message": "Your enrollment drop recommendation was approved and is awaiting application.",
      "data": {
        "academicDecisionId": "507f1f77bcf86cd799439081",
        "studentMemberId": "507f1f77bcf86cd799439071",
        "enrollmentId": "507f1f77bcf86cd799439061",
        "sectionId": "507f1f77bcf86cd799439051"
      },
```

```
      "readAt": null
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

Academic Profile

```
GET /institutions/507f1f77bcf86cd799439011/students/me/academic-profile?
academicYearId=507f1f77bcf86cd799439011
Authorization: Bearer <JWT_TOKEN>
```

```
{
  "academicProfile": {
    "institutionType": "school",
    "student": { "studentMembershipId": "507f1f77bcf86cd799439011", "name": "Sara Ali" },
    "summary": { "complete": false, "hasMissingResults": true },
    "warnings": [{ "code": "MISSING_REQUIRED_RESULTS", "message": "Some required results are
missing." }]
  }
}
```

Postman / Test Artifacts Inspected

Postman collections/environments exist for academic decision workflow, academic profile, academic year, attendance, homework, school academic services, school promotion dashboard, section material, self-enrollment, Stripe billing, repeat policy, responsibility assignments, academic standing, and academic oversight light smoke testing.

Focused tests exist for billing, attendance, homework, section material, authorization/permissions, scoped responsibility management, Head academic oversight, section list permissions, academic profile, academic year, school academic policy/outcomes/report cards/final outcomes/promotion dashboards, university academic policy/profile/outcomes/standing, and self-enrollment.

TODO / VERIFY Summary

- Verify exact patchable `Institution.policies` fields before generated client/schema docs for `PATCH /institutions/:institutionId`.
- Verify destructive delete guards for departments, subjects, sections, and semesters before exposing irreversible UI actions.
- Verify pagination needs for section lists and class promotion summaries as tenant data grows.

- Verify stricter PDF magic-byte validation if upload security requirements increase.
- Verify future date-window overlap handling for responsibility assignments beyond the current duplicate-active guard.
- Verify future billing access gates, customer portal needs, grace periods, trials, and plan limits.
- Verify broader attendance reporting, duplicate-aware risk escalation workflows, and configurable grade-risk thresholds for academic oversight.